https://codehs.com/sandbox/goldstarling0579/java-main-27?filepath=MyProgram.java

Design Doc – Sudoku Project

1. My program generates a fully solved Sudoku board in java (hence the point of this project). My code creates rows, columns, and sub-groups that all contain the numbers 1-9 without repeats. My program follows a simple pattern. It ensures every mini box has the numbers 1-9 first and then shuffles the columns and rows to make sure it fits the requirements of Sudoku.

2a. The 2D array I used is the entire Sudoku Grid itself, where each row and column corresponds to a certain position. The value stored at board[row][col] represents the number in the cell (1-9). Using a 2D array made it easier to access rows and columns by index and loops using nested loops.

2b. I used ArrayLists in three methods when I needed to shuffle. In mixRows() and mixCols(), the ArrayList stored rows or column indexes so they could be shuffled. In mixNums(), the array list held the integers 1-9 to shuffle the arrays. The ArrayList allowed me to swap elements using the .get() and .set() methods, making it easier to sort the Sudoku board.

3. My program generates a Sudoku board by first filling the board itself with integers 1–9. After creating the board, I randomized it by shuffling the rows within each group, shuffling the columns within each group, and finally replacing every value with a new digit to ensure the entire board is valid. This functions because the starting pattern I used already follows all Sudoku rules, and the shuffling only changes the order. Due to this, every row, column, and 3 by 3 box still has the numbers 1 through 9 exactly once, but the board looks different each time the program is run.

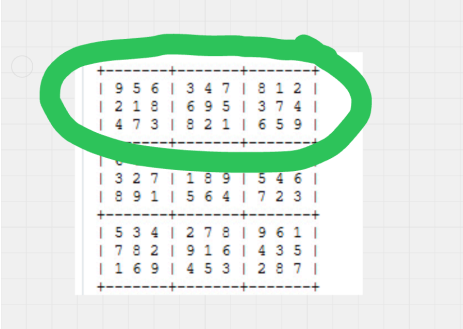
4. Methods:

- **main(String[] args)** is simply where I call all the other methods such as fillBoard(), mixRows(), etc.
- **shuffle(ArrayList<Integer> list)** is a method that shuffles an ArrayList using a while loop. It starts at the last index and continuously swaps the current element with a random earlier element using Math.random().
- **fillBoard()** is a method that fills the 9x9 board by shifting based on row indexes. It produces a starting board where every mini group, column, and row has integers 1-9.
- **mixRows()** is a method that shuffles the three rows within each of the big rows. It uses an ArrayList for row indexes and a temporary 2D array to rearrange the rows.
- **mixCols()** is a method similar to mixRows(). It's a method that shuffles the three columns within each of the big columns. It uses an ArrayList as well for the column indexes and a temporary 1D array to rearrange column values row by row.

- **mixNums()** builds a shuffled version of digits 1-9 using an ArrayList and an array. The code changes each number to a different number using a shuffled list.
- **printBoard()** is a method that prints the board when the program is run. It creates dividers between every 3x3 mini-group created, and a divider printed after every third row to ensure the Sudoku board is neat and not UGLY.

5. To make sure the code I made was doing what I intended it to, I did multiple tests/checks.
- Row check – I printed each row after running the program and checked that every row had exactly the digits 1-9 without repeats. I had to run it multiple times to make sure there would absolutely be no scenario where there would be repeats.
- 3x3 Box check – After fillboard() and shuffling, I checked the nine 3x3 boxes to ensure each one contained integers 1-9 after multiple attempts. This confirmed that the shifts from mixRows() and mixCols() did in fact work.
- Random generation check – I ran the program 10 times to ensure the entire board met the requirements of Sudoku and was different each time. If I would see a repetition in the boards I would see if it was a coincidence for one box or an error in the way I wrote my code.

6. The biggest challenge I faced was creating the Sudoku board without breaking the rules of Sudoku. I knew I needed to shuffle values around; however, if I simply moved rows or columns anywhere I wanted, the board would then be invalid. To solve this problem, I realized that I can swap rows as long as they stay within the same row of big groups (Example of a row in the image below). The board is split into 3 bands, each containing three rows. I realized you can shuffle rows within the same band, but you can't move a row from one band to another. This allowed me to understand what loops I had to write in my methods.



```
+-------+-------+-------+
| 9 5 6 | 3 4 7 | 8 1 2 |
| 2 1 8 | 6 9 5 | 3 7 4 |
| 4 7 3 | 8 2 1 | 6 5 9 |
+-------+-------+-------+
| 3 2 7 | 1 8 9 | 5 4 6 |
| 8 9 1 | 5 6 4 | 7 2 3 |
+-------+-------+-------+
| 5 3 4 | 2 7 8 | 9 6 1 |
| 7 8 2 | 9 1 6 | 4 3 5 |
| 1 6 9 | 4 5 3 | 2 8 7 |
+-------+-------+-------+
```